

Solving the Steiner Tree Problem in graphs with Variable Neighborhood Descent

Matthieu De Laere

KU Leuven KULAK

Dept. of Computer Sciences

matthieu.delaere@student.kuleuven.be

Prof. Patrick De Causmaecker

KU Leuven KULAK

Dept. of Computer Sciences

patrick.decausmaecker@kuleuven.be

San Tu Pham

KU Leuven KULAK

Dept. of Computer Sciences

san.pham@kuleuven.be

Abstract—The Steiner Tree Problem (STP) in graphs is an important problem with various applications in many areas such as design of integrated circuits, evolution theory, networking, etc. In this paper, we propose a algorithm to solve the STP. The algorithm includes a Reducer and a Solver using Variable Neighborhood Descent (VND), interacting with each other during the search. New constructive heuristics and a VND algorithm are proposed. The algorithm is tested on a set of benchmarks which shows encouraging results.

Results obtained by our VND algorithm are promising, especially on computer chip design related graphs and hypercubes.

The rest of the paper is organized as follows. The problem description is presented in section II. Section III gives an overview about the algorithm framework. The two components of the algorithm are presented in section IV and section V. Experimental results are shown in section VI and section VII concludes the paper.

I. INTRODUCTION

The Steiner Tree Problem (STP) is an important problem in combinatorial optimization which has numerous applications, ranging from the design of (very large) integrated circuits to computer networking, evolution theory in biology and more [1]. There are plenty variants of the STP which can be found in [2]. The common part between different variants is the requirement to connect a set of objects with the shortest interconnect possible. In this paper, we investigate the general STP in graphs.

As the STP is $\mathcal{NP}$ -hard [3], most of the work in the literature focuses on non-exact approaches. Borradaile et. al. [4] proposed polynomial time approximation schemes for the STP in Planar Graphs. A Hybrid Local search approach was contributed by Aragao and Ribeiro [5]. Dowsland worked on simulated annealing [6]. An advanced algorithm which includes complicated pre-processing and the combination of Linear Programming formulations and relaxations was proposed by Polzin [7] in his PhD thesis and is the state-of-the-art for the STP.

In this paper, we propose a new algorithm for the STP. The proposed algorithm includes two components: (1) A reducer including a number of tests which simplify the graph by removing nodes and edges that can not be in an optimal solution. (2) A solver which uses a Variable Neighborhood Descent (VND) algorithm. We propose two constructive heuristics to find the initial solution for the VND. For the VND algorithm, two neighborhood structures are defined. During the search, the two components (Reducer and Solver) communicate with each other to improve their performance. In the rest of this paper, while mentioning the VND algorithm, we mean the combination of these two parts.

II. PROBLEM DESCRIPTION

The problem is as follows: Given an undirected graph $G(V, E)$ with a set of nodes V_G and a set of edges E_G (subscript is omitted when the graph is known from the context), find a subgraph S of G with minimal cost which contains at least all nodes of a set $T \subseteq V$ (called terminals). It is allowed to add other nodes than those from T to S and these will be called Steiner nodes. S should be a tree, which means that from every node s and $t \in V_S$ there should exist exactly one path between s and t .

Throughout this paper, certain definitions and notations are used. The cost of an edge between nodes i and j is often referred to as c_{ij} . The degree of a node i is $\delta(i)$ and is equal to the number of edges connected to i . The minimum spanning tree of a graph is denoted by MST. The distance between nodes i and j is denoted with $d(i, j)$. A distance graph D_G of a graph G is a complete graph containing all nodes of G and all edges have a cost equal to $d(i, j)$ in G , for all $i, j \in V$. The terminal distance graph is a subgraph of the distance graph containing only the terminals. With terminal minimum spanning tree (TMST), we denote the MST of the terminal distance graph. The bottleneck of a path is the maximum cost of all edges in the path.

III. ALGORITHM OVERVIEW

The algorithm includes two components communicating and interacting with each other during the search. The first component is the Reducer which includes a number of reduction tests in order to simplify the graph by removing nodes and edges that can not be in an optimal solution. The second component is the Solver which actually produces solutions using a Variable Neighborhood Descent (VND) algorithm. The detailed description of those two components is presented in

section IV and V respectively. During the search, the information about reductions and solutions will be interchanged to improve the performance of both components. Evidently, removing nodes and edges simplifies the task of the solver. In the other direction, some reduction tests can remove more nodes or edges if a tighter upper bound is known.

The detailed framework is as follows. At first, the degree test simplifies the graph from which our constructive heuristic constructs the initial solution. This is a very simple and fast test. It is the only test that is executed before the solver also starts running and constructs an initial solution. This initial solution is then fed to the VND loop. While the initial solution is constructed and the VND runs, the other reduction tests in the Reducer run in parallel. Every time a tighter upper bound is found by the VND, it is fed to the bound-based tests. With this new bound, those tests might be able to reduce the graph furthermore which in turn reduces the search space and hence speeds up the VND. The overview and interaction is illustrated in Figure 1.

IV. REDUCER

An instance of the Steiner Tree Problem can often be reduced using a range of different tests, which are briefly described here. Reductions can be interesting, because a smaller instance can be solved easier. However, time spent on reductions can not be used to search for a solution, so there needs to be a selection in which tests to run. Also important to note is that a reduction does not lose the optimal solution. For details and proofs of these reduction tests, we refer the reader to corresponding publications.

A. General reduction tests

A first series of tests are those that are universally valid and do not depend on an upper or lower bound for the problem. These tests can be run and reduce the problem as soon as the graph is known. The tests that are run in the algorithm described in this paper will be shortly introduced to the reader.

The degree test, described in [8], is the simplest reduction test. This test eliminates nodes from the problem graph by using their degree. For example, a node $i \notin T$ with $\delta(i) = 1$ will never be included in an optimal solution. Imagine a solution containing a Steiner node of degree one, then this solution can be improved upon by removing this node and its incident edge, which reduces the cost. This means that a Steiner node of degree one can never be included in an optimal solution.

The Special Distance Test, introduced by Duin and Volgenant in [9] is more sophisticated. To understand the principals of this test, it is convenient to introduce following terminology: a special path is a path in D_G of which all intermediary vertices (if any) are terminals. The special distance between two vertices i and j is the minimum bottleneck over all special paths from i to j . This test uses the special distance s_{ij} between two nodes i and j and states that if an edge exists between i and j , and $c_{ij} > s_{ij}$, the edge can be removed. The test uses a labeling procedure to find the edges which can be

removed. In its original form, the test is very time consuming. Polzin introduced a simplified but faster version in [7]. In the rest of this paper the Special Distance test refers to this simplified version by Polzin, unless explicitly stated otherwise. For more information about this test, see [7].

Also described by Polzin in [7] is the Triangle Test. This test works fast and can sometimes be very effective. The test determines $\hat{S}$: the cost of the most expensive edge in the TMST, and removes all edges in G with $c_{ij} > \hat{S}$.

B. Bound-based reduction

Another series of reduction tests can be run if an upper bound is known for solutions to the instance of the STP. These tests are used to further reduce the problem after finding a (non-optimal) solution to the given problem.

The Reachability Test, as introduced by Duin in [10] states that given an upper bound B for a solution to the problem, a vertex v can be removed if $\max\{d(v, t)\} > B$, for all $t \in T$. This test can be easily run by first computing the distance from every t to all other nodes, which takes $O(|T||E| + |T||V| \log |V|)$. This order of complexity is obtained by running the Dijkstra algorithm $|T|$ times. With these distances available, the only thing needed is to find the maximum distance for every v , which takes $O(|V||T|)$.

Another test is the Voronoi Region (VR) test, introduced by Polzin in [7]. One can split the graph in different Voronoi regions. Each terminal z defines its own Voronoi region containing all nodes for which this terminal is the closest terminal to the node: $N(z) = \{v \mid d(v, z) < d(v, t), t \in T \setminus z, v \in V\}$. The terminal that defines the Voronoi region is called the base. The radius of a Voronoi region is the distance from its base to the closest node that is not in the Voronoi region: $radius(z) = \min\{d(z, v) \mid v \notin N(z)\}$. We denote the p -th closest terminal to a node v_i by $z_{i,p}$. Assume a graph contains r Voronoi regions, then the radiuses can be ordered as $radius(z_1), \dots, radius(z_r)$ in ascending order. The VR test states that if S is an optimal Steiner tree and v_i is a Steiner node in S , then $d(v_i, z_{i,1}) + d(v_i, z_{i,2}) + \sum_{t=1}^{r-2} radius(z_t)$ is a lower bound for the weight of S . With a certain upper bound B available, it is easy to see that nodes v_i for which $d(v_i, z_{i,1}) + d(v_i, z_{i,2}) + \sum_{t=1}^{r-2} radius(z_t) > B$, v_i can not possibly be in an optimal solution and thus can be removed.

V. VARIABLE NEIGHBORHOOD DESCENT SOLVER

To solve the STP in graphs, a local search algorithm is used. This metaheuristic is often used to solve numerous combinatorial optimization problems [11], [12], [13]. A local search algorithm moves from one possible solution to another one (called the neighbor) by making small changes to the current solution, until no neighbor solution is able to improve the current solution. A neighborhood defines the small changes to be made to move from one solution to another.

Variable Neighborhood Descent (VND) [14] is a local search metaheuristic for solving combinatorial optimization problems which systematically changes neighborhoods throughout the local search procedure. The idea behind VND

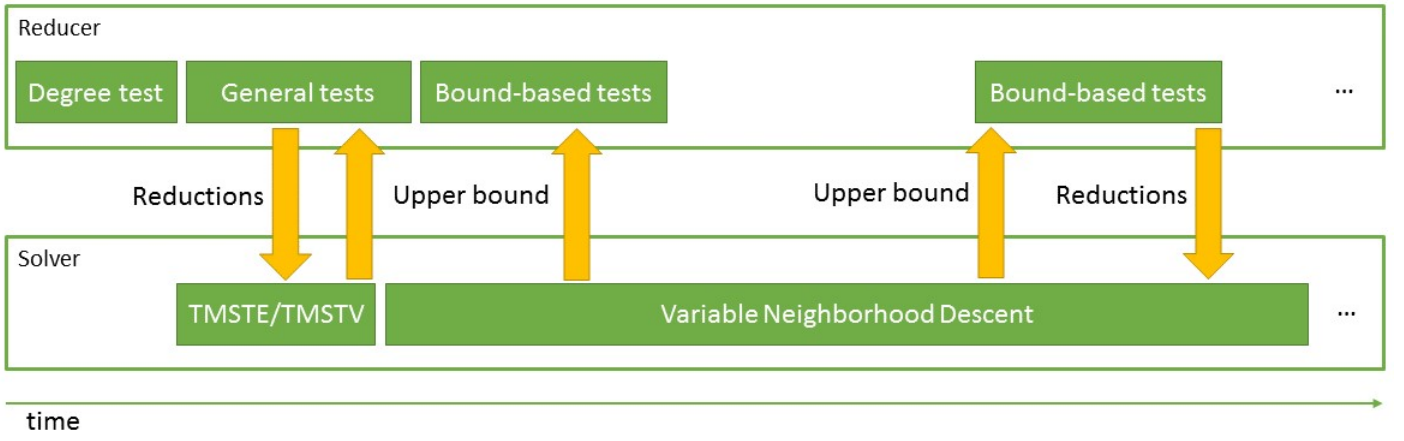


Fig. 1: Overview of the algorithm and the interactions.

is that a local minimum for one neighborhood is not necessarily a local minimum for another neighborhood. The goal of VND is to traverse these local minima by switching to a different neighborhood. We define two neighborhood structures, each of which has different variants regarding to the size of the neighborhood. Before the VND procedure can start, an initial solution is needed. In the below subsections, all components of the VND are presented including the initial solution construction (section V-A), the vertex scores that are utilized in the local search (section V-B) and two local search procedures corresponding to two neighborhood moves (sections V-C, V-D).

Algorithm 1 Variable Neighborhood Descent

Require: Graph $G(V, E)$, set of terminals $T \subseteq V$, minimum and maximum neighborhood size B_{min} and B_{max}

```

 $B \leftarrow B_{min}$ 
 $S \leftarrow InitialSolution()$ 
 $S_{best} \leftarrow S$ 
while true do
   $Ins \leftarrow InsertionLocalSearch(S, B)$ 
   $Rem \leftarrow RemovalLocalSearch(S, B)$ 
   $S \leftarrow \text{Best of } Ins \text{ and } Rem$ 
  if solution has improved then
     $S_{best} \leftarrow S$ 
     $B \leftarrow B_{min}$ 
  else if  $B < B_{max}$  then
     $B \leftarrow B * 2$  // Increase the neighborhood size
  else
    break
  end if
end while

```

A. Initial solution: TMSTE and TMSTV algorithms

Any graph that is a tree and contains all nodes from T would suffice as an initial solution for the local search algorithm to start. This means that using existing algorithms such as

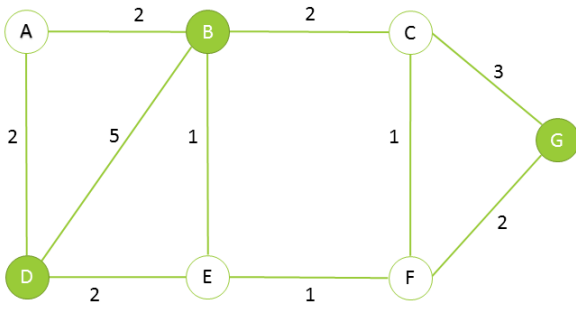
Kruskal's algorithm to find the MST would work and be correct. However, it might be interesting if the initial solution is a bit closer to the optimum. We propose two simple, very similar algorithms that can serve as a starting point for the local search procedure.

The algorithms make use of the following ideas: First is the fact that a minimum spanning tree is a special case of the Steiner Tree Problem, specifically the case in which $T = V$. For this case, fast algorithms exist such as the one by Prim or Kruskal. The second idea is the SDISTG heuristic, as described in [15], which consists of creating the distance graph of the given instance and then taking the MST.

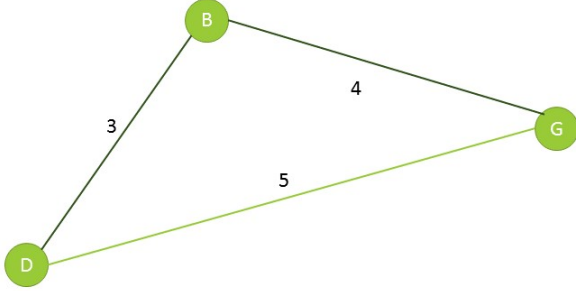
The TMSTE and TMSTV algorithms work as follows: Start by creating the distance graph of only the terminals, and take it's MST. We call this the Terminal Minimum Spanning Tree (TMST). This tree is then expanded with the paths from the original graph. The pruning step is where the two algorithms differ. In the first algorithm all unneeded edges are removed from the original graph, i.e. the edges that are not in the TMST. This is called Terminal Minimum Spanning Tree with Edge pruning (TMSTE). Another possibility is to remove all vertices that are not in the TMST. This is the TMSTV algorithm, for Terminal Minimum Spanning Tree with Vertex pruning. From each of those graphs, we take the MST to remove possible cycles, which leads to two solutions. The best of these two is the starting point for the local search algorithm. The algorithm is illustrated in Figure 2.

B. Vertex scores

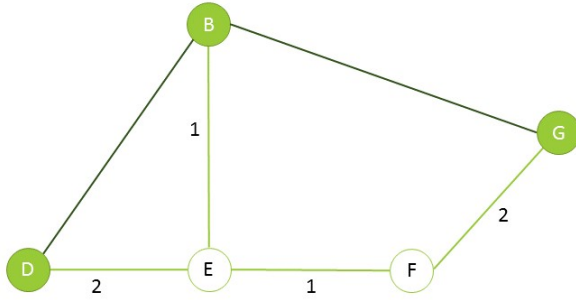
To speed up the local search, the concept of scores is introduced. Each node gets a score which can intuitively be seen as how much a certain node is preferred to be in the solution for the STP. At the initialization, terminals get the highest score and other nodes are neither preferred nor ignored. The score for terminals is immutable. The average score of a node is the average of the last 10 scores that were assigned to the node.



(a) The starting graph for the algorithm.



(b) The terminal distance graph, with its spanning tree indicated.



(c) The expanded TMST, with the tree from the previous step indicated in dark green.

Fig. 2: The TMSTE and TMSTV algorithm.

Throughout the search process the scores are increased if a certain node is able to improve the current solution or decreased if it worsens the solution.

This system of scores is integrated into the local search procedure as follows. During the local search process, each time a new solution is found, the scores of newly inserted nodes will be decreased if the new solution is worse than the previous one. If the new solution is better, the scores of all nodes are increased to also reward nodes that are able to stay in a good solution. To avoid over-scored situations, random restarts are applied to the score system. During the VND procedure, when the maximum neighborhood size is reached, the scores in the graph get a random re-initialization. Five restarts are done before the algorithm stops.

C. Insertion local search

The Steiner node insertion local search is the simplest and works by taking a random starting node i from the current solution with $\delta(i) > 2$ and navigating a path until another node

from the current solution j with $\delta(j) > 2$ is met. Then this path gets inserted in the current solution, while maintaining the tree property. Lastly, the solution is pruned by removing Steiner nodes of degree one. If this leads to a better solution, it is accepted; if not, this new (worse) solution is used to navigate to the next neighbor.

In these neighborhoods the parameter B influences the number of starting nodes, and as a consequence the number of paths added to the graph. The start nodes are taken based on the scores information, i.e. the first $B \ln^2 V$ best-scored nodes are chosen.

Algorithm 2 Insertion local search

Require: Graph $G(V, E)$, set of terminals $T \subseteq V$, current solution $S(V, E)$ and neighborhood size parameter B .

$StartNodes \leftarrow$ Take the $B \ln^2 V$ first nodes i with $\delta(i) > 2$ ordered by descending average score

$S' \leftarrow S$

for all nodes in $StartNodes$ **do**

$P \leftarrow$ Random path to another node s in S with $\delta(s) > 2$

$S' \leftarrow$ Insert edges of P into S' while keeping the three property

Prune nodes with degree one from S'

if Cost of $S' <$ Cost of S **then**

Increase score of all nodes in S'

return S' // Return first improvement

else

Decrease score of all new inserted nodes in S'

end if

end for

return S'

D. Removal local search

The Steiner node removal local search is more time consuming. It removes Steiner nodes from the current solution and then reconnects the disconnected components. This can be done by randomly taking some nodes from each component and running the Dijkstra algorithm to calculate the shortest distance to every other node in the graph. With this information available, a component connecting graph is created in which each component is represented by one node. The edges between those nodes have the cost of the shortest path found in the previous step. After taking the MST of this graph, the solution can be reconnected by expanding these paths and adding them to the disconnected solution.

In these neighborhoods B is the number of Steiner nodes removed at once. Removing multiple Steiner nodes leads to more disconnected components, which in turn leads to more ways to reconnect.

In the Steiner node removal local search the scores are increased in the same way as in the insertion neighborhood. This way of increasing the scores is able to reward the good nodes, as each time they are included in an improved solution they become even more preferred to be in the final solution. If the new solution is worse, the scores of the new nodes that

were needed to reconnect the graph are decreased. Making a move to a neighbor solution in the removal neighborhood is shown in Algorithm 3.

Algorithm 3 Removal local search

Require: Graph $G(V, E)$, set of terminals $T \subseteq V$, current solution $S(V, E)$ and neighborhood size parameter B .
PossibleVictims $\leftarrow$ Take $3B$ first Steiner nodes from S ordered by descending average score
Combinations $\leftarrow$ Take a number of combinations of size B from *PossibleVictims*
for all combinations in *Combinations* **do**
 $S' \leftarrow S$ without all nodes i in combination from S (so S' is a disconnected graph)
 $S' \leftarrow \text{Reconnect}(S')$
 if Cost of $S' <$ Cost of S **then**
 Increase score of all nodes in S'
 return S' // Return first improvement
 else
 Decrease score of all new inserted nodes in S'
 end if
end for

VI. EXPERIMENTAL RESULTS

To test the algorithm, the SteinLib benchmarks [16] were used. The SteinLib library is divided into different test sets. The library has been around for a long time and as such, it is easy to compare the algorithm with others. Each test set in SteinLib is labeled with one of these difficulty ratings:

- **Dead:** These sets of instances have been solved to optimality in various publications. In our results, this category is labeled **D**.
- **Solved:** These sets of instances have been solved to optimality in less than two independent publications. This category is labeled **S**.
- **Unsolved:** These sets of instances have not been solved to optimality in any publication to date. This category is labeled **U**.

The algorithm was implemented in C# with .NET 4.5. The experiments were run on a machine with an Intel Core i7 3632QM CPU and 12GB RAM, running Windows 10. The experiments were run 8 times on each instance to take care of coincidences. The results are shown in Table I. The columns show (from left to right): the problem instance, number of vertices, number of edges, number of terminals, the best, average and worst solution found by the VND algorithm, the average running time to find the solution, the average gap to the optimal solution, the standard deviation on the solutions found and the best known solution. If this best known solution is optimal, it is printed in bold. In the remaining columns the results achieved by Polzin are mentioned, except for the TAQ set as those were not tested on.

The SteinLib library consists of different test sets. Our algorithm was tested on some of these. The first is the E test

set, which contains random generated sparse graphs with edge weights between 1 and 10. The second set is TAQ, containing grid graphs with rectangular holes. These graphs are typically used in VLSI applications (Very-Large-Scale Integration, the design of very large integrated circuits) [17]. The last set is the PUC test set containing hypercubes. For more information about the SteinLib benchmarks, see [16]. The results per instance, grouped per test set, can be found in Table I. In this table, results are compared to those obtained by Polzin [7]. It should be noted that his algorithm is more advanced than the VND algorithm. An overview with time needed and the average gap per test set can be found in Table II. In this summarizing table, only instances for which preprocessing does not lead to an optimum are included as to make it possible to compare with a Hybrid Local Search approach by Aragao and Ribeiro in [5]. The table shows the number of instances for which an optimal solution was found, the average time needed to solve the entire set and the average gap to the optimal for the entire set.

From these results, it is easy to see that the algorithm performs well on the TAQ test set, with an average gap of only 1.15%. It is interesting to note that on this set, the results seem to be quite consistent, which is affirmed by the low standard deviation.

For the E test set, results are slightly worse, with an average gap of 1.9%. This is interesting as this set is considered easier than the TAQ set.

Results on the PUC test set are good, considering the time it takes to achieve them. For instance, when comparing the HC7U instance, the solution provided by Polzin is optimal, but to achieve it he needs almost 5 hours, while our VND algorithm only has a 0.82% gap in just over one minute.

Comparing to the Hybrid Local Search approach in [5], as shown in Table II, it is clear that our VND approach can be improved. Aragao and Ribeiro only report results on the entire sets, and not on individual instances, which makes it difficult to compare performance on particular instances.

VII. CONCLUSION AND FUTURE WORK

In this paper we proposed a Variable Neighborhood Descent algorithm for the STP. Two new constructive heuristics for constructing initial solution were presented. We proposed two neighborhood structures and utilized the concept of scores to guide the local search. The experimental results are promising, especially on hypercube and VLSI-derived instances.

The VND approach works well on some types of graphs (like the hypercubes or VLSI-derived ones), while unable to make any move on others, such as the random complete graphs with Euclidean weights. More research is needed to determine whether the VND approach can be improved to work on all types or graphs.

Future work also includes tuning the parameters used in the VND algorithm. Currently the maximum neighborhood size parameter is fixed at 256 in the insertion neighborhood and 64 for the removal neighborhood. Increasing these parameters evidently increases the running time of the algorithm, but if

Instance	Our VND algorithm									Polzin's algorithm [7]		
	$ V $	$ E $	$ T $	Best	Average	Worst	Time (s)	Gap (%)	Stddev	Best known	Best	Time (s)
E (D)												
E01	2500	3125	5	111	111	111	10.4	0.00	0	111	111	0.5
E02	2500	3125	10	227	227	227	56.5	6.07	0	214	214	0.3
E03	2500	3125	417	4035	4038.3	4043	2556.6	0.63	2.31	4013	4013	0.1
E04	2500	3125	625	5115	5122.2	5137	3819.9	0.42	2.31	5101	5101	0.1
E05	2500	3125	1250	8134	8141.6	8148	9163.4	0.17	4.76	8128	8128	0.1
E06	2500	5000	5	73	75.1	78	22.5	2.89	2.02	73	73	1.2
E07	2500	5000	10	145	150	152	144.2	3.45	2.83	145	145	1.1
E08	2500	5000	417	2666	2679.2	2700	4383.5	1.49	9.44	2640	2640	0.2
E09	2500	5000	625	3634	3645	3662	6103.6	1.14	7.77	3604	3604	0.1
E10	2500	5000	1250	5625	5630	5634	6908.8	0.54	2.53	5600	5600	0.1
E11	2500	12500	5	34	35	37	101.8	2.94	1.41	34	34	0.8
E12	2500	12500	10	68	68	68	184.2	1.49	0	67	67	0.6
E13	2500	12500	417	1307	1313.3	1319	2973.7	2.60	4.35	1280	1280	1.3
E14	2500	12500	625	1758	1760.1	1762	3305.3	1.62	1.6	1732	1732	0.2
E15	2500	12500	1250	2799	2800.4	2805	8341	0.59	2.17	2784	2784	0.2
E16	2500	62500	5	15	15	15	0.8	0.00	0	15	15	0.8
E17	2500	62500	10	25	25.1	26	221.2	0.44	0.31	25	25	0.5
E18	2500	62500	417	597	598.7	602	1391.1	6.15	1.49	564	564	21.6
E19	2500	62500	625	782	783.6	787	1446.4	3.37	1.5	758	758	4.1
E20	2500	62500	1250	1349	1349.9	1353	4336.8	0.59	1.2	1342	1342	0.2
TAQ (D)												
TAQ0014	6466	11046	128	5402	5446.9	5476	4438.8	2.27	23.77	5326	—	—
TAQ0023	572	963	11	623	623	623	13.3	0.32	0	621	—	—
TAQ0365	4186	7074	22	1919	1931.6	1944	653	0.92	7.27	1914	—	—
TAQ0377	6836	11715	136	6544	6611.7	6655	5750.3	3.42	40.66	6393	—	—
TAQ0431	1128	1905	13	897	903	924	59.6	0.67	11.22	897	—	—
TAQ0631	609	932	10	581	584.4	595	11.4	0.59	5.52	581	—	—
TAQ0739	837	1438	16	848	848.7	851	18.5	0.08	1.25	848	—	—
TAQ0741	712	1217	16	852	857.3	862	20.2	1.22	3.74	847	—	—
TAQ0751	1051	1791	16	939	940.8	952	23.3	0.19	4.08	939	—	—
TAQ0903	6163	10490	130	5166	5191.6	5216	3840.8	1.82	15.73	5099	—	—
PUC (U)												
HC6P	64	192	32	4026	4093.9	4133	1.3	2.27	35.45	4003	4003	27.7
HC6U	64	192	32	39	40	42	1.6	2.56	1.05	39	39	13.5
HC7P	128	448	64	7924	8020.6	8106	20.5	1.46	69.10	7905	7905	14362.7
HC7U	128	448	64	77	78.7	80	12.7	2.16	0.82	77	77	17253.5
HC8P	256	1024	128	15680	15888.8	16104	146.2	3.70	114.27	15322	15327	>18000
HC8U	256	1024	128	154	156.9	160	76.2	6.01	2.33	148	148	>18000
HC9P	512	2304	256	31458	31548.8	31757	1198.1	4.32	94.65	30242	30310	>18000
HC9U	512	2304	256	308	310.7	317	512	6.39	2.75	292	292	>18000
HC10P	1024	5120	512	62232	63853.8	63322	7210.9	5.11	337	59797	60679	>18000
HC10U	1024	5120	512	613	633.6	652	2506.4	10.18	12.47	575	581	>18000

TABLE I: Test results for the VND algorithm on different instances, grouped by test set and compared to results from Polzin.

Test set	Instances	Our VND algorithm			GHLS3 [5]		
		Optima	Time (s)	Gap (%)	Optima	Time (s)	Gap (%)
E	19	6	51134.7	1.9	13	1502	0.0093
TAQ	10	4	14829.1	1.15	10	534.8	0
PUC	10	2	11685.8	4.42	—	—	—

TABLE II: Test results for the VND algorithm summarized per test set, only containing the instances which can not be solved by preprocessing, compared to results obtained by Aragao and Ribeiro.

it is guaranteed to lead to better solutions, it might be worth the cost. Another possibility would be to let the algorithm determine the maximum neighborhood size by observing the performance of previous increases. Another parameter that might need some tuning is the number of random restarts, currently arbitrarily fixed at 5. Yet another idea would be to let the algorithm dynamically adapt these values based on previous performance of neighborhood size increases and random restarts.

Lastly, the algorithm was also tested on the $P4E(U)$ test set containing random complete graphs with Euclidean weights. These results are not shown, because the VND algorithm was never able to improve upon the initial solution provided. Future research could investigate the cause or find an appropriate neighborhood that works for this type of graph. Something that might help is Breakout Local Search, as this heuristic has been proven to be working very well on a more constrained version of the STP [18].

REFERENCES

- [1] F. K. Hwang, D. S. Richards, and P. Winter, *The Steiner tree problem*, vol. 53. Elsevier, 1992.
- [2] M. Hauptmann and M. Karpinski, “A Compendium on Steiner Tree Problems,” pp. 1–44, 2013.
- [3] R. M. Karp, *Reducibility among combinatorial problems*. Springer, 1972.
- [4] G. Borradaile, P. Klein, and C. Mathieu, “An $O(n \log n)$ Approximation Scheme for Steiner Tree in Planar Graphs,” vol. V, 1999.
- [5] M. D. Aragao and C. Ribeiro, “Hybrid local search for the Steiner problem in graphs,” *Extended abstracts of . . .*, no. i, pp. 429–434, 2001.
- [6] K. A. Dowsland, “Hill-climbing, simulated annealing and the Steiner problem in graphs,” *Engineering Optimization*, vol. 17, no. 1-2, pp. 91–107, 1991.
- [7] T. Polzin, *Algorithms for the Steiner problem in networks*. PhD thesis, Universitätsbibliothek, 2003.
- [8] T. Koch, T. Koch, and A. Martin, “Solving Steiner Tree Problems in Graphs to Optimality,” vol. 42, no. December, 1996.
- [9] C. W. Duin and A. Volgenant, “An edge elimination test for the Steiner problem in graphs,” vol. 8, pp. 79–83, 1989.
- [10] C. Duin, “Preprocessing the steiner problem in graphs,” in *Advances in Steiner Trees*, pp. 175–233, Springer, 2000.
- [11] C. Voudouris and E. Tsang, “Guided local search and its application to the traveling salesman problem,” *European journal of operational research*, vol. 113, no. 2, pp. 469–499, 1999.
- [12] S. Cai, K. Su, and A. Sattar, “Local search with edge weighting and configuration checking heuristics for minimum vertex cover,” *Artificial Intelligence*, vol. 175, no. 9, pp. 1672–1696, 2011.
- [13] B. Selman, H. J. Levesque, D. G. Mitchell, *et al.*, “A new method for solving hard satisfiability problems,” in *AAAI*, vol. 92, pp. 440–446, 1992.
- [14] N. Mladenović and P. Hansen, “Variable neighborhood search,” *Computers & Operations Research*, vol. 24, no. 11, pp. 1097–1100, 1997.
- [15] S. Voß, “Steiner’s problem in graphs: heuristic methods,” *Discrete Applied Mathematics*, vol. 40, no. 1, pp. 45–72, 1992.
- [16] T. Koch, A. Martin, and S. Voß, *SteinLib: An updated library on Steiner tree problems in graphs*. Springer, 2001.
- [17] M. Jünger, A. Martin, G. Reinelt, and R. Weismantel, “Quadratic 0/1 optimization and a decomposition approach for the placement of electronic circuits,” *Mathematical Programming*, vol. 63, pp. 257–279, 1994.
- [18] Z. H. Fu and J.-K. Hao, “Breakout local search for the Steiner tree problem with revenue, budget and hop constraints,” *European Journal of Operational Research*, vol. 232, no. 1, pp. 209–220, 2014.